

Лабораторная работа № 5

Моделирование с использованием сетей Петри

Нечаева Кира Андреевна

2026-01-01

Содержание

1	Цель работы	5
2	Подготовка рабочего окружения	6
3	Реализация сети Петри	8
4	Стохастическое моделирование	9
5	Детерминированное моделирование	12
6	Анимация работы сети	15
7	Итоговое сравнение	16
8	Параметрическое исследование	17
9	Литературное программирование	20
10	Проверка воспроизводимости	21
11	Структура проекта	23
12	Контроль версий	25
13	Выводы	26
	Приложение А. Quarto-документация основного эксперимента	27
14	Обедающие философы: стохастическая модель	28
14.1	Параметры	29
14.2	Классическая сеть	29
14.3	Сеть с арбитром	31
14.4	Результаты	33
14.5	Вывод	34
	Приложение Б. Quarto-документация параметрического исследования	35
15	Параметрическое исследование	36
15.1	Набор параметров	37
15.2	Серия экспериментов	37
15.3	Полная таблица	40
15.4	Усреднение	40
15.5	График	42
15.6	Вывод	43

Список иллюстраций

2.1	Проверка репозитория и рабочей ветки	6
2.2	Проверка используемых пакетов	7
3.1	Проверка ядра сети Петри	8
4.1	Результат стохастического эксперимента	9
4.2	Стохастическая динамика классической сети	10
4.3	Стохастическая динамика сети с арбитром	11
5.1	Выполнение детерминированного моделирования	12
5.2	Детерминированная динамика классической сети	13
5.3	Детерминированная динамика сети с арбитром	14
6.1	Создание GIF-анимации	15
7.1	Сравнение состояний Eat в двух моделях	16
8.1	Параметрическое исследование	18
8.2	Частота возникновения deadlock	19
9.1	Сгенерированные Quarto-документы и Jupyter Notebook	20
10.1	Успешное выполнение Jupyter Notebook	22
11.1	Итоговая структура проекта	24
12.1	Финальное состояние Git	25

Список таблиц

1 Цель работы

Цель лабораторной работы — изучить применение сетей Петри для моделирования дискретных систем на примере задачи «Обедающие философы».

В работе реализованы классическая сеть Петри и модифицированная сеть с арбитром. Для них проведены стохастическое и детерминированное моделирование, выполнена программная проверка возникновения взаимной блокировки, создана анимация и проведено параметрическое исследование.

2 Подготовка рабочего окружения

Работа выполнялась в Ubuntu через WSL. Проект размещён в общем репозитории лабораторных работ и ведётся в ветке develop.

```
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling$ git pull origin develop
From ssh://gitverse.ru:2222/kanechaeva/2026-1-study-simulation-modeling
 * branch          develop    -> FETCH_HEAD
Already up to date.
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling$ git status
On branch develop
Your branch is up to date with 'origin/develop'.

nothing to commit, working tree clean
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling$ git remote -v
github  git@github.com:kanetschaeva/2026-1-study-simulation-modeling.git (fetch)
github  git@github.com:kanetschaeva/2026-1-study-simulation-modeling.git (push)
origin  ssh://git@gitverse.ru:2222/kanechaeva/2026-1-study-simulation-modeling.git (fetch)
origin  ssh://git@gitverse.ru:2222/kanechaeva/2026-1-study-simulation-modeling.git (push)
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling$ |
```

Рисунок 2.1: Проверка репозитория и рабочей ветки

Для лабораторной работы было создано отдельное окружение Julia и установлены библиотеки для численного моделирования, обработки данных, визуализации, литературного программирования и работы с Jupyter Notebook.

```
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab05/project$ julia --project=. scripts/test_setup.jl
Проверка пакетов:

✓ DrWatson
✓ OrdinaryDiffEq
✓ DataFrames
✓ Plots
✓ CSV
✓ FFMPEG
✓ Literate
✓ IJulia
✓ Quarto
✓ Random
✓ LinearAlgebra
✓ Statistics

Проект: /home/kiran/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab05/project
Каталог данных: /home/kiran/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab05/project/data
Каталог графиков: /home/kiran/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab05/project/plots
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab05/project$ |
```

Рисунок 2.2: Проверка используемых пакетов

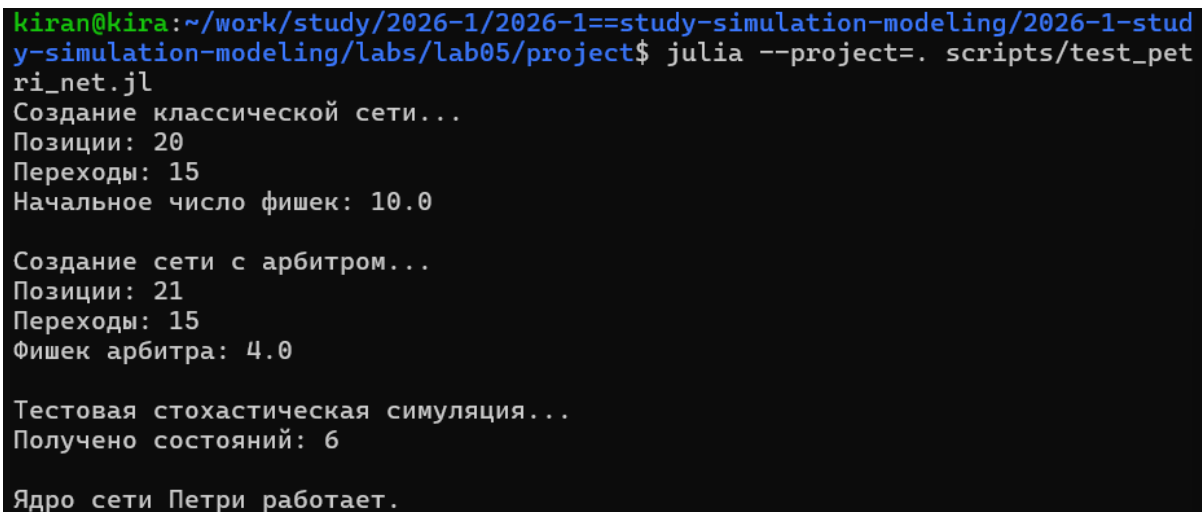
3 Реализация сети Петри

Для представления сети была реализована структура PetriNet, которая хранит количество позиций, переходов, матрицу инцидентности и имена элементов.

Для пяти философов классическая сеть содержит 20 позиций и 15 переходов.

В модели с арбитром появляется дополнительная позиция. В ней изначально находится $N - 1$ фишек, поэтому все философы не могут одновременно начать захват ресурсов.

Перед основными экспериментами была проведена отдельная проверка построения обеих сетей и короткий тест стохастического моделирования.



```
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab05/project$ julia --project=. scripts/test_petri_net.jl
Создание классической сети...
Позиции: 20
Переходы: 15
Начальное число фишек: 10.0

Создание сети с арбитром...
Позиции: 21
Переходы: 15
Фишек арбитра: 4.0

Тестовая стохастическая симуляция...
Получено состояний: 6

Ядро сети Петри работает.
```

Рисунок 3.1: Проверка ядра сети Петри

4 Стохастическое моделирование

Для стохастической модели использован алгоритм Гиллеспи.

Состояние сети меняется отдельными срабатываниями переходов. Выбор следующего перехода и времени его срабатывания зависит от текущего состояния сети.

Классическая сеть и сеть с арбитром запускались с одинаковым значением `seed`. Это позволяет сравнивать модели при одинаковой последовательности псевдослучайных чисел.

После симуляции программа автоматически проверяет доступность переходов. Если ни один переход больше не может сработать, фиксируется `deadlock`.

```
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab05/project$ julia --project=. scripts/dining_philosophers.jl
N = 5
tmax = 50.0

=== Классическая сеть ===
Deadlock обнаружен: true
Последнее время: 6.764

=== Сеть с арбитром ===
Deadlock обнаружен: false
Последнее время: 49.265

Сохранено:
data/dining_classic.csv
data/dining_arbiter.csv
plots/classic_simulation.png
plots/arbiter_simulation.png
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab05/project$ |
```

Рисунок 4.1: Результат стохастического эксперимента

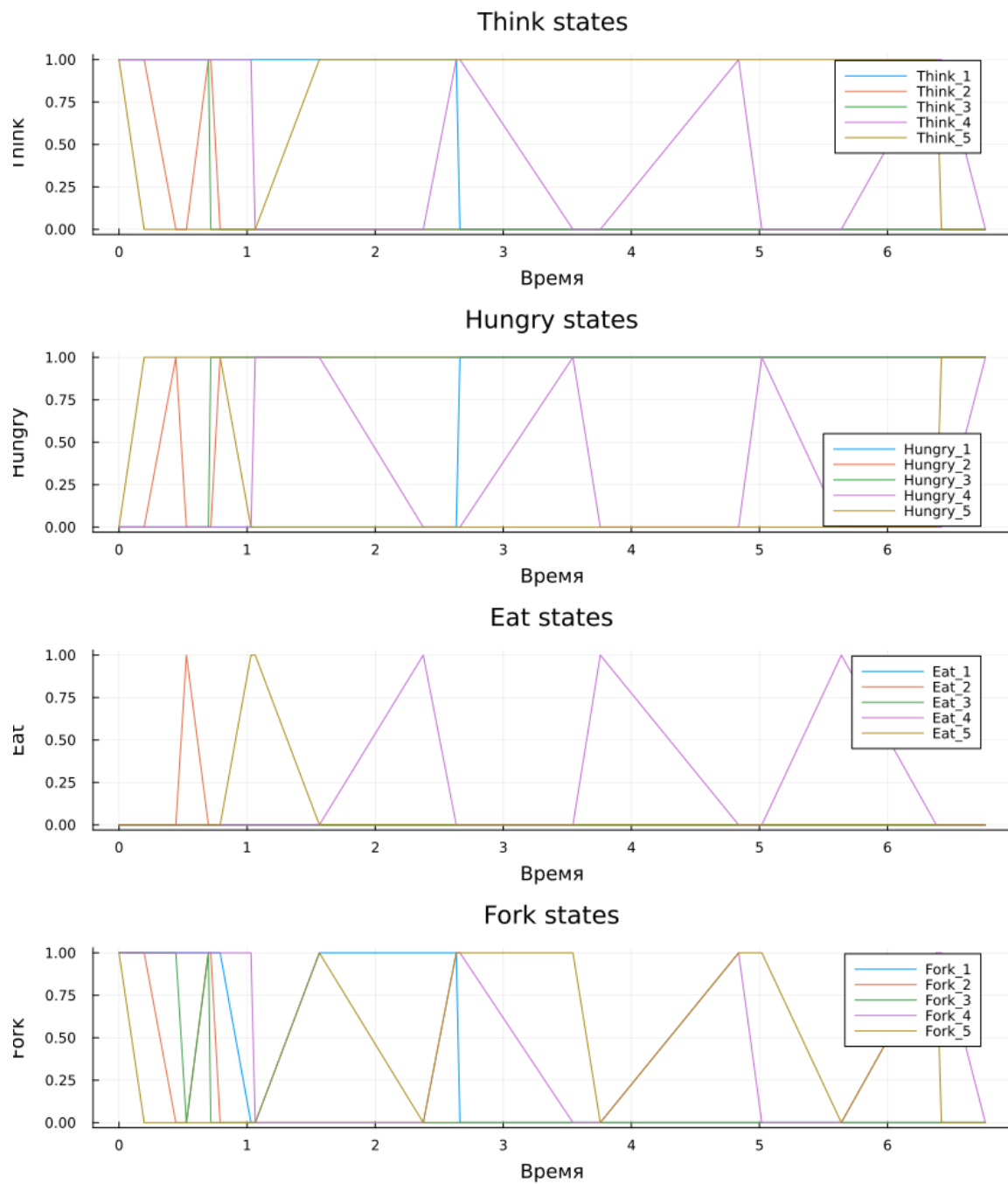


Рисунок 4.2: Стохастическая динамика классической сети

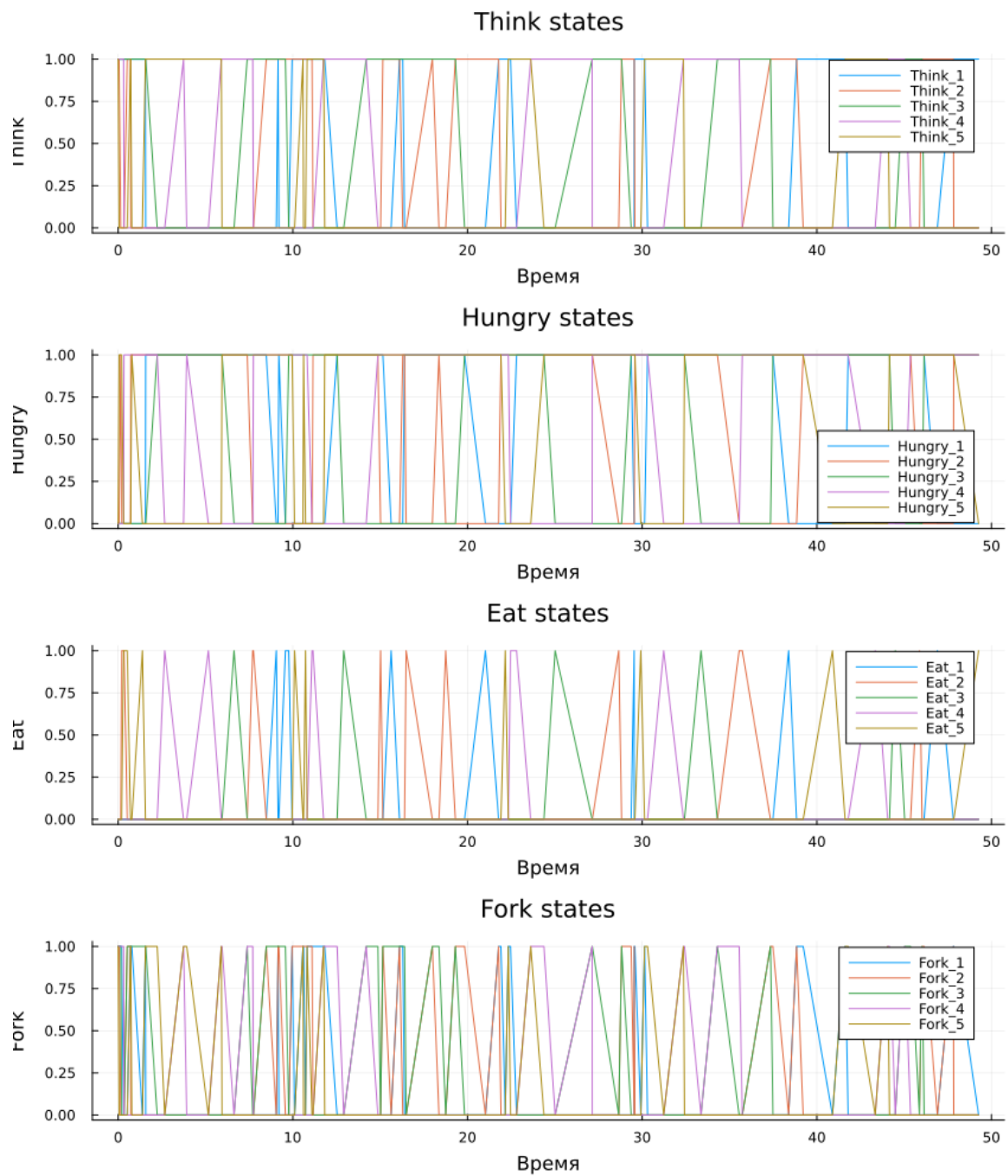


Рисунок 4.3: Стохастическая динамика сети с арбитром

5 Детерминированное моделирование

Дополнительно была исследована детерминированная версия модели.

Матрица инцидентности сети используется для формирования системы дифференциальных уравнений. Поэтому здесь получается непрерывная траектория изменения состояний вместо последовательности отдельных случайных событий.

```
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab05/project$ julia --project=. scripts/dining_philosophers_deterministic.jl
Детерминированная классическая сеть...
Точек траектории: 501

Детерминированная сеть с арбитром...
Точек траектории: 501

Сохранено:
data/dining_classic_ode.csv
data/dining_arbiter_ode.csv
plots/classic_deterministic.png
plots/arbiter_deterministic.png
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab05/project$ |
```

Рисунок 5.1: Выполнение детерминированного моделирования

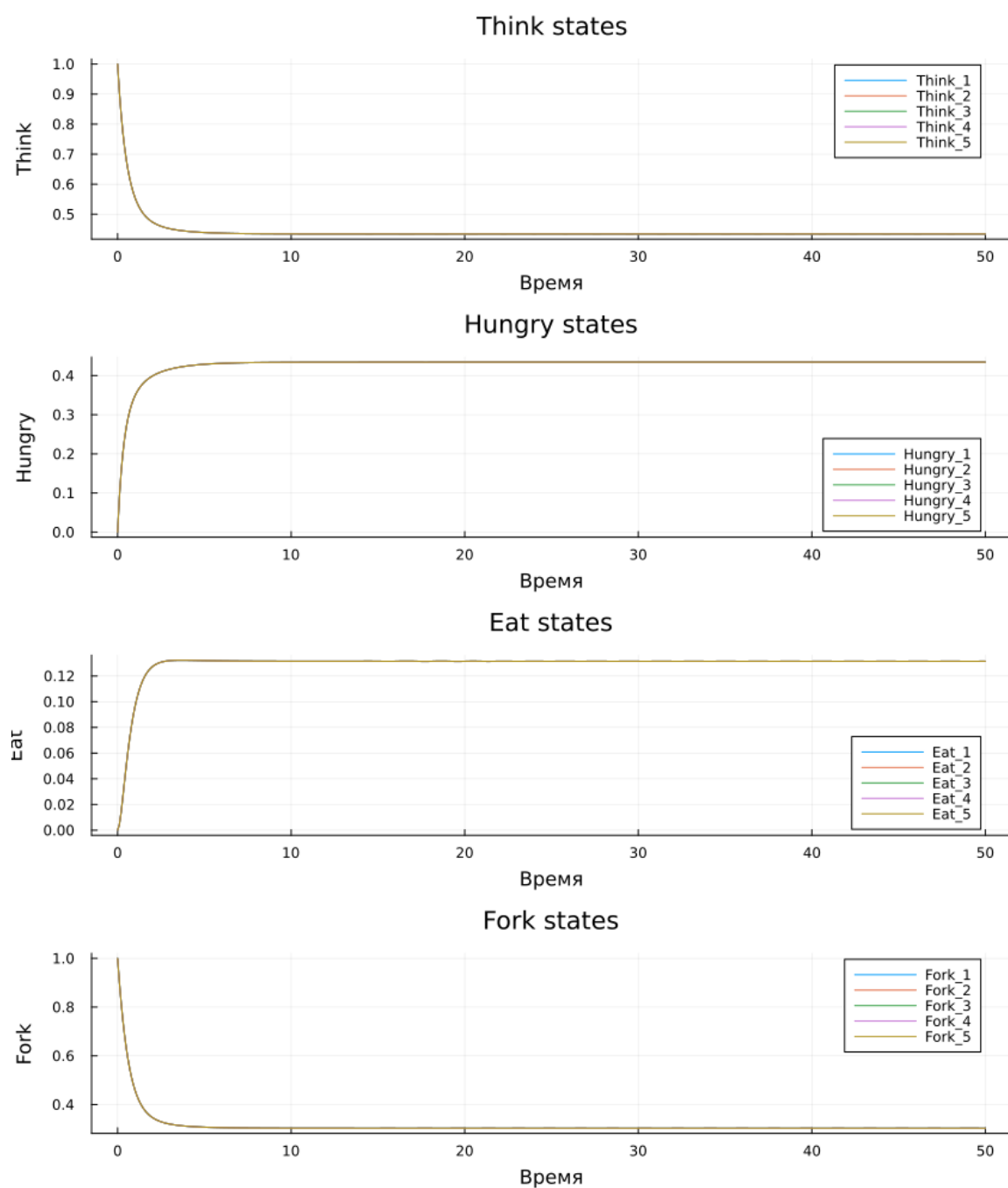


Рисунок 5.2: Детерминированная динамика классической сети

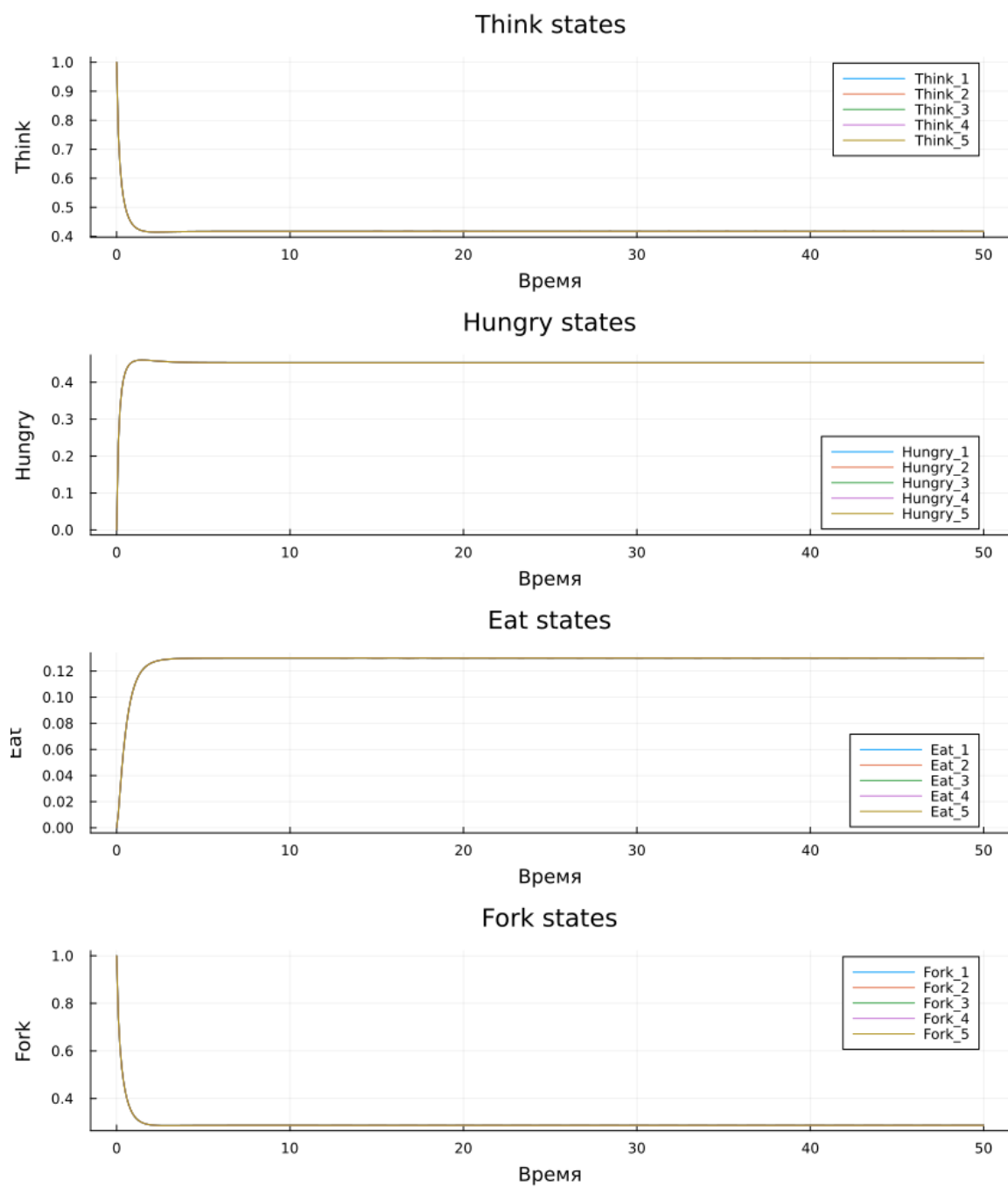


Рисунок 5.3: Детерминированная динамика сети с арбитром

6 Анимация работы сети

Для дополнительной визуализации была создана GIF-анимация.

В анимации использованы три философа. Такое уменьшение количества участников позволяет лучше видеть отдельные позиции и изменение количества фишек.

```
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab05/project$ julia --project=. scripts/dining_philosophers_animation.jl
Получено состояний: 7
[ Info: Saved animation to /home/kiran/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab05/project/plots/philosophers_simulation.gif

Анимация сохранена:
plots/philosophers_simulation.gif
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab05/project$ |
```

Рисунок 6.1: Создание GIF-анимации

Анимация сохранена в файле:

image/plots/philosophers_simulation.gif.

7 Итоговое сравнение

Для итогового сравнения используются ранее сохранённые результаты моделирования.

Скрипт загружает CSV-файлы и анализирует состояния `Eat_i` для всех философов. Повторная симуляция при этом не выполняется.

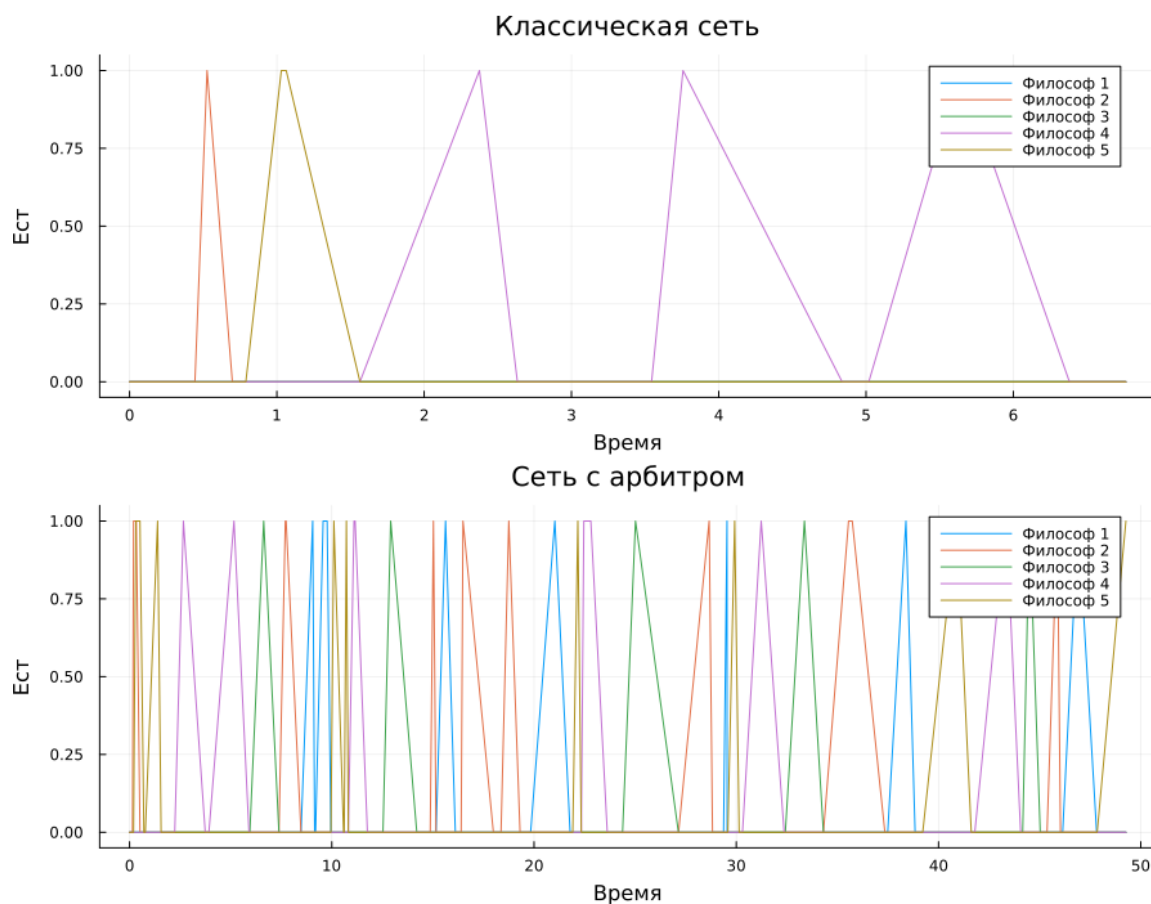


Рисунок 7.1: Сравнение состояний `Eat` в двух моделях

В классической модели длительное отсутствие философов в состоянии `Eat` соответствует взаимной блокировке. В сети с арбитром доступ к ресурсам сохраняется.

8 Параметрическое исследование

Стохастическая модель может давать разные траектории при разных последовательностях случайных чисел.

Поэтому дополнительное исследование проводилось для нескольких параметров:

- $N = 3$;
- $N = 5$;
- $N = 7$;
- нескольких значений `seed`.

Для каждой комбинации параметров отдельно моделировалась классическая сеть и сеть с арбитром.

После этого программа рассчитывала долю запусков, завершившихся взаимной блокировкой.

```

kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab05/project$ julia --project=. scripts/dining_philosophers__param.jl
Параметрическое исследование...
N=3, seed=101, classic=true, arbiter=false
N=3, seed=202, classic=true, arbiter=false
N=3, seed=303, classic=true, arbiter=false
N=3, seed=404, classic=true, arbiter=false
N=3, seed=505, classic=true, arbiter=false
N=5, seed=101, classic=true, arbiter=false
N=5, seed=202, classic=true, arbiter=false
N=5, seed=303, classic=true, arbiter=false
N=5, seed=404, classic=true, arbiter=false
N=5, seed=505, classic=true, arbiter=false
N=7, seed=101, classic=true, arbiter=false
N=7, seed=202, classic=true, arbiter=false
N=7, seed=303, classic=true, arbiter=false
N=7, seed=404, classic=true, arbiter=false
N=7, seed=505, classic=true, arbiter=false

Сводная таблица:
3x3 DataFrame
  Row  |  N      classic_deadlock_rate  arbiter_deadlock_rate
      |  Int64  Float64                Float64
-----+-----
  1    |    3          1.0                0.0
  2    |    5          1.0                0.0
  3    |    7          1.0                0.0

Сохранено:
data/deadlock_parameters.csv
data/deadlock_parameters_summary.csv
plots/deadlock_frequency_params.png
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab05/project$ |

```

Рисунок 8.1: Параметрическое исследование

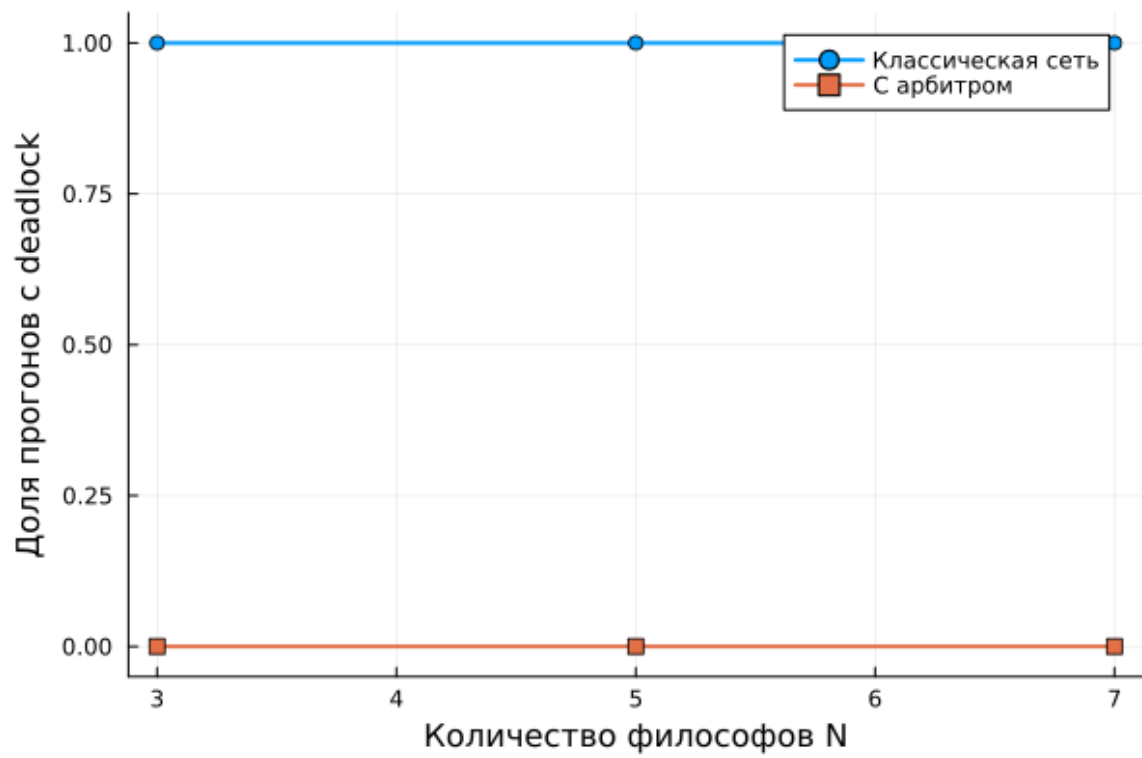


Рисунок 8.2: Частота возникновения deadlock

Использование нескольких повторных запусков позволяет оценивать не отдельную случайную траекторию, а устойчивость результата.

9 Литературное программирование

Основные вычислительные сценарии были оформлены в литературном стиле.

Из одного исходного файла автоматически создаются:

- чистый Julia-код;
- Jupyter Notebook;
- документация Quarto.

Это позволяет поддерживать код и пояснение в одном источнике.

```
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab05/project$ find markdown -type f -name "*.qmd" | sort
markdown/dining_philosophers/dining_philosophers.qmd
markdown/dining_philosophers_animation/dining_philosophers_animation.qmd
markdown/dining_philosophers_deterministic/dining_philosophers_deterministic.qmd
markdown/dining_philosophers_report/dining_philosophers_report.qmd
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab05/project$ find notebooks -type f -name "*.ipynb" | sort
notebooks/dining_philosophers/dining_philosophers.ipynb
notebooks/dining_philosophers_animation/dining_philosophers_animation.ipynb
notebooks/dining_philosophers_deterministic/dining_philosophers_deterministic.ipynb
notebooks/dining_philosophers_report/dining_philosophers_report.ipynb
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab05/project$ |
```

Рисунок 9.1: Сгенерированные Quarto-документы и Jupyter Notebook

10 Проверка воспроизводимости

Автоматически созданные Jupyter Notebook были отдельно выполнены.

Это проверяет, что сгенерированные notebooks содержат все необходимые команды и могут самостоятельно воспроизвести вычислительные эксперименты.

```

Starting kernel event loops.
[NbConvertApp] Writing 6862 bytes to notebooks/dining_philosophers_deterministic/dining_philosophers_deterministic.ipynb
ГOTOB0: notebooks/dining_philosophers_deterministic/dining_philosophers_deterministic.ipynb

=====
Выполняется: notebooks/dining_philosophers_animation/dining_philosophers_animation.ipynb
=====
[NbConvertApp] Converting notebook notebooks/dining_philosophers_animation/dining_philosophers_animation.ipynb to notebook
Starting kernel event loops.
[NbConvertApp] Writing 5945 bytes to notebooks/dining_philosophers_animation/dining_philosophers_animation.ipynb
ГOTOB0: notebooks/dining_philosophers_animation/dining_philosophers_animation.ipynb

=====
Выполняется: notebooks/dining_philosophers_report/dining_philosophers_report.ipynb
=====
[NbConvertApp] Converting notebook notebooks/dining_philosophers_report/dining_philosophers_report.ipynb to notebook
Starting kernel event loops.
[NbConvertApp] Writing 433891 bytes to notebooks/dining_philosophers_report/dining_philosophers_report.ipynb
ГOTOB0: notebooks/dining_philosophers_report/dining_philosophers_report.ipynb

=====
Выполняется: notebooks/dining_philosophers__param/dining_philosophers__param.ipynb
=====
[NbConvertApp] Converting notebook notebooks/dining_philosophers__param/dining_philosophers__param.ipynb to notebook
Starting kernel event loops.
[NbConvertApp] Writing 11776 bytes to notebooks/dining_philosophers__param/dining_philosophers__param.ipynb
ГOTOB0: notebooks/dining_philosophers__param/dining_philosophers__param.ipynb
(jupyter) kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling/labs/lab05/project$ 5

```

Рисунок 10.1: Успешное выполнение Jupyter Notebook

11 Структура проекта

После выполнения работы проект содержит исходный модуль сети Петри, литературные программы, сгенерированные версии программ, notebooks, документацию, таблицы, графики и анимацию.

```
y-simulation-modeling/labs/lab05/project$ find src scripts markdown notebook
s data plots -maxdepth 2 -type f | sort
data/deadlock_parameters.csv
data/deadlock_parameters_summary.csv
data/dining_arbiter.csv
data/dining_arbiter_ode.csv
data/dining_classic.csv
data/dining_classic_ode.csv
markdown/dining_philosophers/dining_philosophers.qmd
markdown/dining_philosophers__param/dining_philosophers__param.qmd
markdown/dining_philosophers_animation/dining_philosophers_animation.qmd
markdown/dining_philosophers_deterministic/dining_philosophers_deterministic
.qmd
markdown/dining_philosophers_report/dining_philosophers_report.qmd
notebooks/dining_philosophers/dining_philosophers.ipynb
notebooks/dining_philosophers__param/dining_philosophers__param.ipynb
notebooks/dining_philosophers_animation/dining_philosophers_animation.ipynb
notebooks/dining_philosophers_deterministic/dining_philosophers_deterministi
c.ipynb
notebooks/dining_philosophers_report/dining_philosophers_report.ipynb
plots/arbiter_deterministic.png
plots/arbiter_simulation.png
plots/classic_deterministic.png
plots/classic_simulation.png
plots/deadlock_frequency_params.png
plots/final_report.png
plots/philosophers_simulation.gif
scripts/dining_philosophers.jl
scripts/dining_philosophers/dining_philosophers.jl
scripts/dining_philosophers__param.jl
scripts/dining_philosophers__param/dining_philosophers__param.jl
scripts/dining_philosophers_animation.jl
scripts/dining_philosophers_animation/dining_philosophers_animation.jl
scripts/dining_philosophers_deterministic.jl
scripts/dining_philosophers_deterministic/dining_philosophers_deterministic.
jl
scripts/dining_philosophers_report.jl
scripts/dining_philosophers_report/dining_philosophers_report.jl
scripts/intro.jl
scripts/tangle.jl
scripts/test_petri_net.jl
scripts/test_setup.jl
src/DiningPhilosophers.jl
```

Рисунок 11.1: Итоговая структура проекта

12 Контроль версий

Все изменения были сохранены в Git и отправлены в удалённые репозитории.

```
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling$ git status
On branch develop
Your branch is up to date with 'origin/develop'.

nothing to commit, working tree clean
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling$ git log --oneline --decorate --graph -6
* d9df236 (HEAD -> develop, origin/develop, github/develop) feat(lab05): add Petri net dining philosophers model
* a032a6b docs(lab04): add report and presentation
* 12c661d feat(lab04): add agent-based SIR model and experiments
* ef66194 docs(lab03): add report and presentation
* 37bdaa4 feat(lab03): add Daisyworld agent-based model
* 3df7cb2 docs(lab02): add generated parameter files
kiran@kira:~/work/study/2026-1/2026-1==study-simulation-modeling/2026-1-study-simulation-modeling$ |
```

Рисунок 12.1: Финальное состояние Git

13 Выводы

В лабораторной работе была реализована задача «Обедающие философы» с использованием аппарата сетей Петри.

Были построены классическая сеть и сеть с дополнительным арбитром. Для обеих моделей выполнено стохастическое и детерминированное моделирование.

Программная проверка состояния сети позволяет автоматически обнаруживать взаимную блокировку. Добавление арбитра изменяет правило распределения ресурсов и предотвращает ситуацию, когда все философы одновременно удерживают только одну вилку и ожидают вторую.

Дополнительно было проведено параметрическое исследование для различного числа философов и нескольких случайных траекторий.

Все основные вычисления оформлены в литературном стиле, а автоматически созданные Jupyter Notebook отдельно выполнены для проверки воспроизводимости.

Приложение А. Quarto-документация основного эксперимента

Ниже интегрирована статическая версия документации Quarto, автоматически созданной из литературного исходного кода.

14 Обедающие философы:

стохастическая модель

Автор: Нечаева Кира Андреевна

Группа: НКНбд-01-23

Сравниваются классическая сеть Петри и сеть с дополнительным арбитром.

```
ENV["GKSwstype"] = "100"

using DrWatson
@quickactivate "project"

include(
    srcdir(
        "DiningPhilosophers.jl"
    )
)

using .DiningPhilosophers

using DataFrames
using CSV
using Plots
using Random
```

14.1 Параметры

```
N =  
    5  
  
tmax =  
    50.0  
  
seed =  
    123  
  
println(  
    "N = ",  
    N  
)  
  
println(  
    "tmax = ",  
    tmax  
)
```

14.2 Классическая сеть

```
println(  
    "\n=== XXXXXXXXXXXXXXX XXXX ==="  
)  
  
net_classic,  
u0_classic,  
_ =  
    build_classical_network(  
        N
```

```

    )

df_classic =
    simulate_stochastic(
        net_classic,
        u0_classic,
        tmax;
        rng=MersenneTwister(
            seed
        )
    )

CSV.write(
    datadir(
        "dining_classic.csv"
    ),
    df_classic
)

dead_classic =
    detect_deadlock(
        df_classic,
        net_classic
    )

println(
    "Deadlock ██████████: ",
    dead_classic
)

println(
    "██████████ ██████: ",
    round(

```

```

        df_classic.time[end],
        digits=3
    )
)

plot_classic =
    plot_marking_evolution(
        df_classic,
        N
    )

savefig(
    plot_classic,
    plotsdir(
        "classic_simulation.png"
    )
)

```

14.3 Сеть с арбитром

```

println(
    "\n===  XXXX  X  XXXXXXXXXX  ==="
)

net_arbiter,
u0_arbiter,
_ =
    build_arbiter_network(
        N
    )

df_arbiter =

```

```

simulate_stochastic(
    net_arbiter,
    u0_arbiter,
    tmax;
    rng=MersenneTwister(
        seed
    )
)

CSV.write(
    datadir(
        "dining_arbiter.csv"
    ),
    df_arbiter
)

dead_arbiter =
    detect_deadlock(
        df_arbiter,
        net_arbiter
    )

println(
    "Deadlock ██████████: ",
    dead_arbiter
)

println(
    "██████████ █████: ",
    round(
        df_arbiter.time[end],
        digits=3
    )
)

```

```

)

plot_arbiter =
    plot_marking_evolution(
        df_arbiter,
        N
    )

savefig(
    plot_arbiter,
    plotsdir(
        "arbiter_simulation.png"
    )
)

```

14.4 Результаты

```

println(
    "\nXXXXXXXXXX: "
)

println(
    "data/dining_classic.csv"
)

println(
    "data/dining_arbiter.csv"
)

println(
    "plots/classic_simulation.png"
)

```

```
println(  
    "plots/arbiter_simulation.png"  
)
```

14.5 Вывод

Классическая сеть допускает взаимную блокировку. Арбитр ограничивает число философов, которые могут одновременно начать захват вилок.

Приложение Б. Quarto-документация параметрического исследования

Ниже интегрирована документация параметрической версии программы.

15 Параметрическое исследование

Автор: Нечаева Кира Андреевна

Группа: НКНбд-01-23

Исследуется частота deadlock при различном количестве философов и нескольких случайных seed.

```
ENV["GKSwstype"] = "100"

using DrWatson
@quickactivate "project"

include(
    srcdir(
        "DiningPhilosophers.jl"
    )
)

using .DiningPhilosophers

using DataFrames
using CSV
using Plots
using Random
using Statistics
```

15.1 Набор параметров

```
N_values =  
    [3, 5, 7]  
  
seeds =  
    [  
        101,  
        202,  
        303,  
        404,  
        505  
    ]  
  
tmax =  
    50.0  
  
results =  
    NamedTuple[]  
  
println(  
    "XXXXXXXXXXXXXXXX XXXXXXXXXXXXXXXX..."  
)
```

15.2 Серия экспериментов

```
for N in N_values  
  
    for seed in seeds  
  
        net_classic,  
        u0_classic,
```

```

_ =
    build_classical_network(
        N
    )

df_classic =
    simulate_stochastic(
        net_classic,
        u0_classic,
        tmax;
        rng=MersenneTwister(
            seed
        )
    )

dead_classic =
    detect_deadlock(
        df_classic,
        net_classic
    )

net_arbiter,
u0_arbiter,
_ =
    build_arbiter_network(
        N
    )

df_arbiter =
    simulate_stochastic(
        net_arbiter,
        u0_arbiter,
        tmax;

```

```

        rng=MersenneTwister(
            seed
        )
    )

dead_arbiter =
    detect_deadlock(
        df_arbiter,
        net_arbiter
    )

push!(
    results,
    (
        N=N,
        seed=seed,
        classic_deadlock=
            dead_classic,
        arbiter_deadlock=
            dead_arbiter,
        classic_last_time=
            df_classic.time[end],
        arbiter_last_time=
            df_arbiter.time[end]
    )
)

println(
    "N=",
    N,
    ", seed=",
    seed,
    ", classic=",

```

```

        dead_classic,
        ", arbiter=",
        dead_arbiter
    )
end
end

```

15.3 Полная таблица

```

df =
    DataFrame(
        results
    )

CSV.write(
    datadir(
        "deadlock_parameters.csv"
    ),
    df
)

```

15.4 Усреднение

```

summary =
    combine(
        groupby(
            df,
            :N
        ),
        :classic_deadlock =>
            (

```

```

        x ->
            mean(
                Float64.(
                    x
                )
            )
    ) =>
        :classic_deadlock_rate,

    :arbiter_deadlock =>
        (
            x ->
                mean(
                    Float64.(
                        x
                    )
                )
            ) =>
                :arbiter_deadlock_rate
        )

CSV.write(
    datadir(
        "deadlock_parameters_summary.csv"
    ),
    summary
)

println(
    "\nXXXXXXXX XXXXXXXX:"
)

show(

```

```

summary;
allrows=true,
allcols=true
)

println()

```

15.5 График

```

p =
    plot(
        summary.N,
        summary.classic_deadlock_rate;
        marker=:circle,
        linewidth=2,
        xlabel="XXXXXXXXXX XXXXXXXXX N",
        ylabel="XXXX XXXXXXXXX X deadlock",
        label="XXXXXXXXXXXXXXX XXXX",
        ylim=(-0.05, 1.05)
    )

plot!(
    p,
    summary.N,
    summary.arbiter_deadlock_rate;
    marker=:square,
    linewidth=2,
    label="X XXXXXXXXXX"
)

savefig(
    p,

```

```

    plotsdir(
        "deadlock_frequency_params.png"
    )
)

println(
    "\nXXXXXXXXXX: "
)

println(
    "data/deadlock_parameters.csv"
)

println(
    "data/deadlock_parameters_summary.csv"
)

println(
    "plots/deadlock_frequency_params.png"
)

```

15.6 Вывод

Повторные прогоны позволяют оценивать устойчивость результата относительно случайной траектории алгоритма Гиллеспи.